

Quiz Engine

1. Feature Name

Quiz Engine

2. Purpose

- **Problem Solved:** The Quiz Engine addresses the need to assess student comprehension and validate module mastery before allowing students to progress through a learning path.
- **Why It Exists:** It ensures academic accountability and guides the learning process by gatekeeping module completion behind a customizable passing score rather than simple read-acknowledgment.
- **Role in Application:** It acts as the final evaluation checkpoint for learning path modules. It coordinates with question banks (uploaded as JSON files), implements grading logic, tracks attempt counts, logs user metrics, and updates student module completion status.

3. What the User Can Do

Student

- **Access instructions & constraints:** View quiz parameters, including the time limit, passing percentage, maximum allowed attempts, and difficulty filters.
- **Start quiz attempts:** Initiate new attempts (up to the maximum attempt count).
- **Take the quiz:** Navigate through selected questions, choose answers, flag questions for later review, clear selections, and track elapsed time via a countdown timer.
- **Submit attempts:** Submit answers manually, or have them auto-submitted when the timer expires.
- **View results:** Instantly check attempt metrics (percentage score, time spent, correct answer counts, and pass/fail status).
- **Review answers:** Examine a detailed breakdown of a submitted attempt, viewing all questions, selected vs. correct options, and explanations.

Instructor

- **Search Question Banks:** Search and inspect active question banks to identify content for quizzes.

- **Manage Quizzes:** Create and configure quizzes (Title, question count, difficulty, random/sequential selection, time limit, passing percentage, maximum attempts).
- **Publish & Archive Quizzes:** Manage quiz life-cycles (Draft, Published, Archived) to control student visibility.
- **Link Quizzes to Modules:** Assign published quizzes to modules within their owned learning paths.

Admin

- **Manage Question Banks:** Upload new JSON question banks, update versions, download bank configurations, and delete, archive, or restore question banks.
- **Delete Quizzes:** Permanently remove quizzes from the system.
- **Access Quiz Analytics:** View global analytics (average scores, pass rate, score distribution, and question-level success metrics).

4. Feature Workflow

[Student View Module]

- Clicks "Start Quiz"
- GET `api/v1/modules/{moduleId}/quiz` (Retrieve assignment detail)
- GET `api/v1/quizzes/{quizId}` (Retrieve quiz parameters)
- POST `api/v1/quizzes/{quizId}/attempt?moduleId={moduleId}` (Initialize attempt)
- Attempt is generated with a random seed and saved **in** InProgress state
- Redirects to Quiz Attempt UI with randomized questions & shuffled options
- Student selects answers
- PUT `api/v1/attempts/{attemptId}/answer` (Saves answer **for** each question dynamically)
- Student submits (or timer expires)
- POST `api/v1/attempts/{attemptId}/submit` (Grade attempt, update progress)
- Backend compares selections, calculates score, and updates module status
- Redirects to Quiz Result Page showing pass/fail status and percentage
- Clicks "Review Answers"
- GET `api/v1/attempts/{attemptId}/review` (Generates review view with correct options & explanations)



5. How It Works Internally

Question-Bank Selection

Instructors or admins create quizzes by associating them with a specific question bank. When creating or updating a quiz via `QuizService.CreateAsync()` or `QuizService.UpdateAsync()`, the backend validates that the requested `QuestionCount` does not exceed the available questions within the target `QuestionBank` (and checks matching counts against the difficulty filter if one is applied).

Question Selection

Question selection is performed dynamically inside `AttemptService.GetSelectedQuestionsAsync()` in `backend/Services/Attempt/AttemptService.cs`.

- It fetches all questions belonging to the configured `QuestionBankId`.
- If `DifficultyFilter` is configured, it appends a difficulty constraint: `.Where(q => q.Difficulty == quiz.DifficultyFilter.Value)`.
- It orders the questions by `q.Id` to establish a stable base list.
- If `SelectionMode` is `SelectionMode.Random (0)`, it shuffles the list using the attempt's unique seed and returns `Take(quiz.QuestionCount)`.
- If `SelectionMode` is `SelectionMode.Sequential (1)`, it takes the first `quiz.QuestionCount` questions without shuffling.

Question Randomization

Randomization uses the Fisher-Yates shuffle algorithm implemented in `AttemptService.ShuffleList<T>()`. It creates a local pseudo-random number generator: `var rng = new Random(seed);` The seed used is `QuizAttempt.RandomSeed`, which is generated during attempt creation in `AttemptService.StartAttemptAsync()` using `Random.Shared.Next()`. Because this seed is persisted in the database, the order of randomized questions is stable across page reloads and multiple requests for the same attempt.

Option Ordering

Option randomization occurs in both `BuildAttemptResponse()` and `GetReviewAsync()` in `backend/Services/Attempt/AttemptService.cs`. It shuffles options for each question using: `ShuffleList(q.Options.ToList(), attempt.RandomSeed + q.Id)` Using `attempt.RandomSeed + q.Id` as a deterministic seed guarantees that the options are shuffled uniquely per question and attempt, while remaining stable upon reloading the attempt.

Attempt Creation

Initiated via `AttemptService.StartAttemptAsync()`.

1. Verifies the quiz is published and assigned to the specified module.

2. Checks for an active attempt (`InProgress` or `Created`). If one exists, it is returned to resume progress.
3. If no active attempt is found, it evaluates the total attempts against `MaximumAttempts` . If the limit is reached, it throws an error. If the user has already passed, it blocks attempt creation.
4. Creates a new `QuizAttempt` entity in the `InProgress` status, sets `RandomSeed` and `AttemptNumber` , updates the module status to `QuizAttempted` , and writes a `QUIZ_STARTED` audit log.

Answer Submission & Validation

Saved dynamically using `AttemptService.SaveAnswerAsync()` via `PUT` `api/v1/attempt/{attemptId}/answer` .

- Checks if the attempt is active (`InProgress` or `Created`) and owned by the requester.
- If an answer already exists for the given question, it updates the `OptionId` and resets `AnsweredAt` .
- Otherwise, it inserts a new `StudentAnswer` entity.

Score Calculation & Passing Percentage

Performed in `AttemptService.SubmitAttemptAsync()` during submission.

1. Determines which questions were selected for this attempt using `GetSelectedQuestionsAsync(quiz, attempt.RandomSeed)` .
2. Iterates over the selected questions, checking if the student's answered `OptionId` matches the correct option (`o.IsCorrect == true`).
3. Calculates `Percentage = Math.Round((decimal)correctCount / totalQuestions * 100, 2)` .
4. Marks `Passed = attempt.Percentage >= quiz.PassingPercentage` .
5. Updates status to `AttemptStatus.Evaluated` , sets `SubmittedAt` , and logs audit events (`QUIZ_COMPLETED` and `QUIZ_PASSED` / `QUIZ_FAILED`).
6. If `Passed` is true, updates the module status to `Completed` and marks the module complete in `ProgressService` .

Attempt Tracking

Attempts are tracked inside `QuizAttempt` and verified at initialization. A database index is maintained on `[QuizId, UserId, AttemptNumber]` to speed up attempt evaluation queries.

Result Generation

Returns the total score, total questions, percentage, passed status, time spent, and attempt number in `SubmitResponseDto` to direct the student's completion overview UI.

Review Generation

Handled by `AttemptService.GetReviewAsync()`. It recreates the selected question order and option layouts using the stored `RandomSeed`, resolves correct vs. selected options, and pairs them with question explanations for step-by-step student feedback.

6. Frontend Implementation

Layer	File Path	Responsibility
Page	<code>frontend/src/pages/quiz/QuizInstructionsPage.tsx</code>	Displays quiz instructions, parameters, and initiates the attempt.
Page	<code>frontend/src/pages/quiz/QuizAttemptPage.tsx</code>	Main interface for taking the quiz, containing the question palette, timer, flag triggers, and navigation.
Page	<code>frontend/src/pages/quiz/QuizResultPage.tsx</code>	Displays final scores, completion status (pass/fail), attempt statistics, and redirection links.
Page	<code>frontend/src/pages/quiz/QuizReviewPage.tsx</code>	Provides step-by-step breakdown of questions, highlighting user responses, correct options, and explanations.
Page	<code>frontend/src/pages/admin/AdminQuizEditorPage.tsx</code>	Handles assignment and removal of published quizzes to/from modules.
Page	<code>frontend/src/pages/admin/AdminQuizManagementPage.tsx</code>	CRUD controller interface for instructors/admins to design and publish quizzes.

Layer	File Path	Responsibility
Page	frontend/src/pages/admin/AdminQuizAnalyticsPage.tsx	Visualizes global scores, pass rates, score distribution, and question-level success rates.
Page	frontend/src/pages/admin/AdminQuestionBanksPage.tsx	Admin panel to upload, update, download, archive, restore, and delete question bank files.
Service	frontend/src/services/quizService.ts	Dispatches API calls for quiz management, linking, attempts, saves, submissions, and analytics.
Service	frontend/src/services/questionBankService.ts	Dispatches API calls for uploading and managing question bank files.

7. Backend Implementation

Layer	File Path	Responsibility
Controller	backend/Controllers/QuizController.cs	Endpoints for quiz definition CRUD, publishing states, and module mappings.
Controller	backend/Controllers/AttemptController.cs	Student endpoints to start, save answers, submit, and review attempts.
Controller	backend/Controllers/QuestionBankController.cs	Administrative endpoints to upload,

Layer	File Path	Responsibility
		download, and manage question banks.
Controller	backend/Controllers/AnalyticsController.cs	Endpoints to fetch user progress analytics and quiz-specific performance reports.
Service	backend/Services/Quiz/QuizService.cs	Controls business rules for quiz lifecycles, configuration boundaries, and module association.
Service	backend/Services/Attempt/AttemptService.cs	Logic for starting attempts, Fisher-Yates randomization, saving answers, grading, and review assembly.
Service	backend/Services/QuestionBank/QuestionBankService.cs	Handles question bank persistence, version control, and JSON parsing.
Service	backend/Services/Validation/JsonValidationService.cs	Performs strict schema verification, duplicate detection, and file constraints on uploaded question banks.
Service	backend/Services/Analytics/AnalyticsService.cs	Aggregates attempt percentages, range distributions, and question failure statistics.
Service	backend/Services/Progress/ProgressService.cs	Updates student completion logs and updates parent learning

Layer	File Path	Responsibility
		path progress upon passing a quiz.

8. Database Implementation

Entities and Relationships

Entity/Table	Purpose	Important Relationship
Quizzes	Stores quiz configurations and grading criteria.	Linked to <code>QuestionBanks</code> (Restrict) and <code>ModuleQuizzes</code> .
Questions	Individual evaluation questions parsed from JSON files.	Linked to <code>QuestionBanks</code> (Cascade) and <code>Options</code> (Cascade).
Options	Multiple-choice options linked to questions.	Linked to <code>Questions</code> (Cascade).
QuestionBanks	Raw container and source of truth for evaluation questions.	Linked to <code>Questions</code> and <code>Quizzes</code> .
QuizAttempts	Evaluated attempt records for individual students.	Linked to <code>Quizzes</code> (Restrict), <code>Modules</code> (Restrict), <code>Users</code> (Restrict), and <code>StudentAnswers</code> (Cascade).
StudentAnswers	Tracked answer selections submitted by students.	Linked to <code>QuizAttempts</code> (Cascade), <code>Questions</code> (Restrict), and <code>Options</code> (Restrict).
ModuleQuizzes	Mappings link a module with a single quiz.	Linked to <code>Modules</code> (Cascade) and <code>Quizzes</code> (Restrict).

Important Database Constraints

- `StudentAnswers` contains a unique index on `[QuizAttemptId, QuestionId]`, preventing duplicate selections for the same question.

- `QuizAttempts` maintains an index on `[QuizId, UserId, AttemptNumber]` to quickly fetch history.
- `ModuleQuizzes` maintains a unique index on `ModuleId`, restricting each module to a maximum of one quiz.

9. Security & Authorization

- **Role-Based Security:** Endpoints are guarded with role-based policies (`[Authorize]`).
 - `Admin` only: Delete quizzes, upload/restore/delete question banks, download bank source JSON, and access quiz analytics.
 - `Admin, Instructor`: Create/edit quizzes, publish/unpublish quizzes, and assign/remove quizzes to/from modules. Instructors are restricted to managing quizzes they created.
 - `Student` (No role restrictions): Start attempts, save answers, submit, view results, and review attempts.
- **Ownership Checks:**
 - Instructors can only retrieve, update, publish, or delete quizzes that match their user ID (`CreatedById == userId`).
 - Quizzes can only be assigned to modules if the instructor owns the parent learning path (`module.LearningPath.CreatedById == userId`).
 - Attempt details, answer saving, submissions, and reviews verify that `attempt.UserId == userId`, preventing students from accessing or modifying other students' attempts.

10. Important Business Rules

1. **Unique Quiz Title:** Duplicate quiz titles are blocked by the database.
2. **Status Flow Constraints:** Quizzes can only be assigned to modules or attempted by students if they are in the `Published` status. Deleted or Archived quizzes are blocked.
3. **Question Bank Upload Validation:**
 - File size must be ≤ 5 MB.
 - Must have a `.json` extension.
 - Root fields (`Title`, `Subject`, `Questions`) are mandatory.
 - Each question must have ≥ 2 options, a correct answer matching one option, a valid difficulty, an explanation, and unique text within the upload.
4. **Quiz Creation Limits:** The quiz `QuestionCount` must be greater than 0 and cannot exceed the total questions in the bank. If a difficulty filter is set, the count cannot exceed the total questions matching that difficulty.

5. Attempt Constraints:

- If a student has already passed a quiz, they cannot start a new attempt.
- If a student reaches the maximum attempts limit without passing, they are blocked from starting new attempts.

6. **Progress Updates:** If a student passes a quiz, the parent module's status changes to `Completed` in both the `Modules` and `Progresses` tables, triggering path completion updates.

7. **Linked Quizzes Retention:** Quizzes cannot be deleted while assigned to any module (`ModuleQuizzes.Count > 0`).

11. Example of Internal Execution

Step 1: Initialize Quiz Attempt

- **User Action:** Student clicks "Start Quiz" on the Instructions page.
- **Frontend:** Dispatches `POST /api/v1/quizzes/12/attempt?moduleId=5`.
- **Controller:** `AttemptController.StartAttempt` receives the call and forwards it to `AttemptService.StartAttemptAsync(12, 5, userId)`.
- **Service & DB:**
 - Fetches Quiz #12 and checks if it is published.
 - Verifies that the user has not exceeded 3 attempts and has not passed.
 - Retrieves 5 questions under the question bank with `RandomSeed = 10478129`.
 - Creates a new `QuizAttempt` row in the database.
- **Response:** Returns `AttemptStartResponseDto` with the randomized question sequence and shuffled options.

Step 2: Answer Question

- **User Action:** Student selects Option #45 for Question #8.
- **Frontend:** Dispatches `PUT /api/v1/attempts/34/answer` with body `{ questionId: 8, optionId: 45 }`.
- **Controller:** `AttemptController.SaveAnswer` receives the call and invokes `AttemptService.SaveAnswerAsync(34, requestDto, userId)`.
- **Service & DB:** Updates or inserts a row in `StudentAnswers` setting `OptionId = 45` and `AnsweredAt = DateTime.UtcNow`.
- **Response:** Returns `200 OK`.

Step 3: Submit Attempt

- **User Action:** Student clicks "Submit Quiz".
- **Frontend:** Dispatches `POST /api/v1/attempts/34/submit`.

- **Controller:** `AttemptController.SubmitAttempt` receives the call and invokes `AttemptService.SubmitAttemptAsync(34, userId)`.
- **Service & DB:**
 - Recomputes the randomized questions list using seed `10478129`.
 - Compares the student's selected option ID for each question against the option marked `IsCorrect = true`.
 - Calculates a final score (e.g., 4/5 correct = 80%).
 - Determines `Passed = true` (80% >= 40% passing percentage).
 - Sets attempt status to `Evaluated` and records elapsed time.
 - Updates the student's module progress status to `Completed`.
- **Response:** Returns `SubmitResponseDto` showing score, percentage, and passed status.
- **Frontend UI:** Redirects the user to the result overview page.

12. Files Involved

Frontend

- `frontend/src/pages/quiz/QuizInstructionsPage.tsx`
- `frontend/src/pages/quiz/QuizAttemptPage.tsx`
- `frontend/src/pages/quiz/QuizResultPage.tsx`
- `frontend/src/pages/quiz/QuizReviewPage.tsx`
- `frontend/src/pages/admin/AdminQuizEditorPage.tsx`
- `frontend/src/pages/admin/AdminQuizManagementPage.tsx`
- `frontend/src/pages/admin/AdminQuizAnalyticsPage.tsx`
- `frontend/src/pages/admin/AdminQuestionBanksPage.tsx`
- `frontend/src/services/quizService.ts`
- `frontend/src/services/questionBankService.ts`
- `frontend/src/types/quiz.types.ts`
- `frontend/src/types/questionBank.types.ts`

Backend

- `backend/Controllers/QuizController.cs`
- `backend/Controllers/AttemptController.cs`
- `backend/Controllers/QuestionBankController.cs`
- `backend/Controllers/AnalyticsController.cs`
- `backend/Interfaces/Services/IQuizService.cs`
- `backend/Interfaces/Services/IAttemptService.cs`
- `backend/Interfaces/Services/IQuestionBankService.cs`
- `backend/Interfaces/Services/IValidationService.cs`

- backend/Interfaces/Services/IAalyticsService.cs
- backend/Services/Quiz/QuizService.cs
- backend/Services/Attempt/AttemptService.cs
- backend/Services/QuestionBank/QuestionBankService.cs
- backend/Services/Validation/JsonValidationService.cs
- backend/Services/Analytics/AnalyticsService.cs
- backend/Services/Progress/ProgressService.cs

Database (EF Core Configurations)

- backend/Entities/Quiz.cs
- backend/Entities/QuizAttempt.cs
- backend/Entities/StudentAnswer.cs
- backend/Entities/QuestionBank.cs
- backend/Entities/Question.cs
- backend/Entities/Option.cs
- backend/Entities/ModuleQuiz.cs
- backend/Configurations/QuizConfiguration.cs
- backend/Configurations/QuizAttemptConfiguration.cs
- backend/Configurations/ModuleQuizConfiguration.cs
- backend/Configurations/StudentAnswerConfiguration.cs

13. Functionality

- **Upload Question Bank:** Admins can upload JSON files detailing subject-matter questions.
- **Version Control:** Automatically increments versions when uploading question banks with matching title/subject keys.
- **Question Bank Management:** Archiving, restoring, downloading, and deleting question banks.
- **Quiz Creation & Settings:** Configure parameter boundaries (Difficulty filters, selection modes, passing margins, time allowances, and attempt counts).
- **Quiz Lifecycle Management:** State controls to transition quizzes through Draft, Published, and Archived stages.
- **Module Assignment:** Bind published quizzes to learning path modules.
- **Randomization & Selection:** Randomize question sequencing and option layouts using Fisher-Yates with persisted attempt seeds.
- **Attempt Resumption:** Resume unfinished active attempts if the browser is reloaded or closed.

- **Dynamic Response Saving:** Dynamic, question-by-question response saving during attempts.
- **Score Calculations:** Real-time grading and percentage scoring upon submission.
- **Automatic Progress Tracking:** Automatically updates student module states and completion histories upon passing attempts.
- **Answer Audits & Reviews:** Review correct vs. selected answers alongside step-by-step explanations.
- **Performance Analytics:** Visual performance indicators tracking attempt rates, distributions, and problem questions.